

Actividad de recuperación día 8



PROTOTIPO INTERACTIVO: TABLA, BUSCADOR, TECLADO Y VALIDACIÓN

Fecha	Lunes 28 de septiembre de 2026
Aprendiz	Sneider Alvarez Santos
Ficha	3428392
Documento	1016717046
Programa	Desarrollo web — SENA
Proyecto	La Carambola Dorada — sistema de gestión de salones de billar
Publicación	Servidor local XAMPP — http://localhost/plan-mejoramiento-desarrollo-web/

Bitácora del día: qué hice

- 1 La tabla de productos se pinta desde `js/datos-prueba.js`. Borré las siete filas que estaban escritas a mano y el `<tbody>` quedó vacío en el HTML.
- 2 Buscador en vivo: un `input` y un `filter`, sin botón de buscar. Filtra mientras se escribe.
- 3 El botón del menú pasó de `<label>` a `<button>` en las tres páginas, y con eso ya responde a la tecla Enter. Conectado en `app/menu.js`.
- 4 Validación del formulario de producto con el mensaje **junto al campo**, en un `<p>` debajo de cada campo. Ningún `alert`.

Archivos que creé y archivos que toqué

ARCHIVO	ESTADO	QUÉ HACE
app/menu.js	Nuevo	Abre y cierra el menú lateral, y mantiene al día el <code>aria-expanded</code> . Lo cargan las tres páginas.
app/productos.js	Nuevo	Pinta la tabla, filtra con el buscador, y valida el formulario.
productos.html	Modificado	Sin filas escritas, buscador, cuatro huecos de mensaje, y los tres <code><script></code> .
dashboard.html	Modificado	Solo el menú: <code><button></code> en lugar de <code><label></code> , y el <code><script></code> .
componentes.html	Modificado	Igual que el tablero.
css/estilos.css	Modificado	<code>.boton-mini</code> , <code>.boton-peligro</code> , <code>.campo-error</code> y el ajuste de la tabla.
js/datos-prueba.js	Sin tocar	El del día 7. Solo se carga; los datos se copian para poder editarlos.

El código de la aplicación va en `app/` y los datos del cliente en `js/`, que es como el día 3 definió las carpetas del proyecto.

Punto 1. La tabla se dibuja con JavaScript

En el día 6 la tabla tenía siete filas escritas a mano. Ahora el `<tbody>` llega vacío y las 16 filas las arma `app/productos.js` a partir del archivo de datos.

```
<tbody id="tabla-productos-body"></tbody>
```

Cada fila se construye con `createElement` y su texto se pone con `textContent`. Se podría haber escrito la fila con `innerHTML` y quedaría más corto, pero el formulario deja añadir productos: el nombre que escriba el usuario acaba dentro de una celda. Con `innerHTML`, un nombre con `<img onerror=...>` se ejecutaría. Con `textContent` siempre se trata como texto.

Un detalle que no es opcional: cada `<td>` lleva su `data-label`. En móvil la hoja de estilos convierte cada celda en «etiqueta: valor» con `content: attr(data-label)`, así que sin ese atributo las celdas saldrían sin su nombre en el celular. La primera vez que probé sin ponerlo, las etiquetas quedaron pegadas a la izquierda y sin texto.

El código de cada producto va en un `<th scope="row">` y las otras cinco columnas en `<td>`, igual que estaban escritas a mano. El precio se formatea con el mismo `Intl.NumberFormat` del día 7.

Punto 2. El buscador filtra mientras se escribe

No hay botón de buscar: el evento `input` filtra y vuelve a pintar en cada tecla, y también al pegar o al borrar. La búsqueda mira tres campos: nombre, categoría y código, para que «tiza» encuentre la tiza y «consumible» encuentre los cinco consumibles.

```
const texto = normalizar(buscador.value);
if (!texto) return lista;
return lista.filter((producto) =>
```

```

normalizar(producto.nombre).includes(texto) ||
normalizar(producto.categoria).includes(texto) ||
normalizar(producto.codigo).includes(texto)
);

```

La función `normalizar` quita las tildes antes de comparar, así que escribir «extension» sin tilde encuentra «Extensión de taco» y «Conector de extensión».

Hay dos detalles de accesibilidad:

- El contador de resultados lleva `role="status"`, para que un lector de pantalla anuncie «5 de 16 productos coinciden con la búsqueda» al filtrar.
- Cuando no hay nada, aparece un aviso con la clase `.alerta` que dice que no coincidió ninguna producto. Se quita con el atributo `hidden`.

Punto 3. El menú también responde a la tecla Enter

Este punto tenía un problema escondido. En el día 6 el botón del menú era un `<label for="interruptor-menu">`, y un `<label>` **no es un control**: no recibe el foco, no se puede tabular hasta él y la tecla Enter no lo activa. El menú se abría con el ratón y no con el teclado. Cambiar el `<label>` por un `<button>` es justamente lo que lo arregla.

```

<button type="button" class="boton-menu" id="boton-menu"
  aria-expanded="false" aria-controls="menu-lateral">
  <span class="boton-menu__icono" aria-hidden="true"></span>
  <span class="boton-menu__texto">Menú</span>
</button>

```

Por qué no hay ningún `keydown` en el código: un `<button>` ya emite `click` cuando se presiona Enter. Si además se escuchara `keydown`, la tecla contaría dos veces, una por cada evento, y el menú se abriría y se cerraría en el mismo instante. Así que el archivo solo escucha `click` y la tecla funciona sola.

El `<input type="checkbox">` del día 6 **sigue siendo el estado**. El botón real no reescribe el CSS: marca y desmarca la misma casilla, y el selector `.interruptor-menu:checked ~ .panel__menu` abre el menú igual que antes. Así el menú sigue funcionando aunque el JavaScript no cargue. Además se marca `aria-expanded`, que es para lo que esa regla del CSS ya existía desde el día 6 sin que nadie la usara.

El botón **Cerrar** también era un `<label>` y por lo mismo tampoco respondía al teclado. Si solo hubiera cambiado el de abrir, el que abriera el menú con Enter no lo podría cerrar con el teclado. Así que los dos son `<button>` y los dos funcionan con Enter.

El botón del menú solo existe a lo ancho de un celular: a partir de 1024 px se oculta con `display: none`, porque en escritorio el menú lateral ya está siempre a la vista. Por eso las pruebas de Enter se hicieron a 360 px y no a 1440 px.

Delegación de eventos sobre la tabla

Cada fila lleva sus botones **Editar** y **Eliminar**, y hay **un solo escucha para toda la tabla**, puesto en el `<tbody>`:

```

cuerpo.addEventListener("click", (evento) => {
  const boton = evento.target.closest("button[data-accion]");
  if (!boton) return;

```

```
const accion = boton.dataset.accion;
const codigo = boton.dataset.codigo;
if (accion === "editar") abrirFormulario(codigo);
else if (accion === "eliminar") borrar(codigo);
});
```

Si se le pusiera un `addEventListener` a cada botón al pintar, esos escuchas morirían con las filas: el buscador borra y recrea el contenido en cada tecla, y los botones nuevos no tendrían quién atienda el clic. Como el `<tbody>` no cambia nunca, un solo escucha suyo sirve para las dieciséis filas, para las cinco que queden filtradas y para las que se agreguen después.

Usé `data-codigo` y no `data-id` como en el ejemplo del enunciado: los datos de este proyecto no tienen un identificador numérico, su clave es el código del producto, así que los botones llevan el código y se comparan como texto.

Editar carga el producto en el formulario que ya había y deja el código bloqueado, porque el código es la clave con la que se localiza la fila: si se pudiera cambiar, el producto perdería su clave y el botón Editar dejaría de encontrarlo. **Eliminar** quita la fila y avisa en la misma página, sin ventana de confirmación ni `alert`.

Punto 4. Los errores aparecen junto al campo

El formulario lleva `novalidate` desde antes de este día, así que el navegador no pinta su burbuja propia. Los avisos que se ven son los míos de la página, un `<p class="campo-error">` debajo de cada campo, con `role="alert"` para que un lector de pantalla lo pronuncie.

CAMPO	REGLA QUE PIDE EL ENUNCIADO	EN EL HTML	SI EL CAMPO ESTÁ VACÍO
nombre	Mínimo tres caracteres	<code>required</code> <code>minlength="3"</code>	«Escribe el nombre del producto.»
valor_alquiler	Mayor que cero	<code>required min="0.01"</code> <code>step="0.01"</code>	«Escribe el valor de alquiler.»
cantidad	Entero no negativo	<code>required min="0"</code> <code>step="1"</code>	«Escribe cuántas existencias hay.»
categoria	Seleccionada	<code>required</code>	«Selecciona una categoría.»

Cada regla trae **dos mensajes**: uno para el campo vacío y otro para el campo lleno con un valor que no vale. La diferencia son dos cadenas de texto, pero importa: decirle «el valor debe ser mayor que cero» a un campo que está vacío confunde, porque no es un valor, no hay nada.

Para «entero no negativo» uso una expresión regular y no `Number()`, porque `Number("3.5")` da `3.5`, que no es un entero, y `Number(" ")` da `0`, que sí lo sería. Con `/^\d+$/` el `0` se acepta, el `-3` se rechaza y el `2.5` también.

Además de mostrarse, cada campo que falla se marca con `aria-invalid="true"` y apunta a su mensaje con `aria-describedby`. Los dos atributos se quitan solos en cuanto el campo se corrige, y el mensaje desaparece porque el `<p>` se queda vacío. Con `:empty` no hace falta esconderlo con JavaScript, y así el elemento nunca sale del HTML, que es lo que necesita el lector de pantalla.

Los mensajes no salen mientras se escribe por primera vez: el formulario espera al primer intento de guardado. A partir de ahí, cada tecla vuelve a revisar, y el error se borra solo en cuanto se corrige.

Los otros dos campos obligatorios. El código y la fecha de ingreso también son obligatorios desde el principio, pero el enunciado solo pide validar cuatro. Si aún faltan, el formulario se niega a guardar y avisa en la página, en vez de dejar enviar un producto sin código. No se llamó a `reportValidity()` a propósito, porque esa función abre la burbuja del navegador y el enunciado pide los avisos junto al campo.

Los criterios de evaluación, punto por punto

CRITERIO	ESTADO	CÓMO SE COMPROBÓ
No existe ningún <code>onclick</code> en el HTML	CUMPLE	En las cuatro páginas de la aplicación hay 0 atributos <code>onclick</code> , <code>onchange</code> , <code>onsubmit</code> e <code>oninput</code> . Solo aparece la palabra dentro de esta bitácora, escrita en prosa.
La tabla usa delegación de eventos	CUMPLE	Un solo <code>addEventListener</code> en el <code><tbody></code> , con <code>closest("button[data-accion]")</code> . Verificado con clics reales del ratón sobre 16 filas distintas.
Los errores se muestran junto al campo	CUMPLE	Se midió la distancia vertical entre cada campo y su mensaje: los cuatro están a menos de 40 px, o sea pegados a su campo. Y no hay ningún <code>alert</code> , <code>confirm</code> ni <code>prompt</code> en el proyecto.
No cumple si la validación se resuelve con ventanas <code>alert</code>	EVITADO	El <code>alert</code> se substituyó por una variable durante la prueba y se confirmó que nunca se llama.

Los problemas que me salieron y cómo los resolví

La séptima columna desbordó la página

La columna de Acciones hizo que la tabla pidiera 31 px más de los que hay a 768 px, y la página se desbordó en horizontal a 768 px y a 1024 px. Se arregló con dos cosas: bajar el padding horizontal de las celdas de `0.9rem` a `0.7rem` a partir de 768 px, y poner `overflow-x: auto` en `.tabla-scroll` como red de seguridad. Con eso, si algún día el contenido vuelve a pedir más ancho, lo que scrollea es la tabla dentro de su caja y no la página entera.

Tres fallos que eran de mi prueba y no del código

La primera vez que corrí las comprobaciones salieron ocho fallos y tres no eran culpa del código:

- Buscé «categoria» sin tilde y salían 0 filas. Pensé que el filtro fallaba, pero **ninguna categoría lleva tilde**: son «Consumible», «Repuesto» y «Implemento de juego». La prueba estaba mal planteada. Ahora se prueba con «extension», que sí encuentra los dos productos que llevan «extensión» en el nombre.
- El menú no se abría con Enter. Lo probaba a 1440 px, pero el botón del menú tiene `display: none` a partir de 1024 px. A ese ancho no hay nada que abrir. Ahora se prueba a 360 px.

- Un producto nuevo no se guardaba. Me faltaba llenar la fecha de ingreso en la prueba, y como es obligatoria el formulario se negaba a guardarlo. **Ahí el código actuó bien:** fue la prueba la que estaba incompleta.

La tecla Enter no quería activarse

Con la primera manera de mandar la tecla, el `<button>` no se activaba. Resultó ser un problema de cómo se envía la tecla, no del código de la página: hay que mandar la bajada de tecla (`rawKeyDown`) y después el carácter (`char` con un retorno de carro), no las dos cosas juntas, porque así el navegador lo toma como inserción de texto y no como activación.

Un fallo de escritura en el código que dejó sin revisar

Se me colaron tres palabras en inglés o en chino dentro de los comentarios de `app/menu.js` y `app/productos.js`. No rompen nada, pero en un trabajo que se lee quedan mal. Se buscaron y se quitaron.

Comó se verificó

No me fié de abrir la página y mirar. Se midió con Edge controlado por CDP, mandando teclas y clics de verdad.

QUÉ SE MIDIÓ	RESULTADO
Comprobaciones en el navegador real	53 de 53 , 0 fallos. Consola sin un solo error ni excepción.
La tabla pintada	16 filas, 16 códigos distintos, 32 botones de acción, ninguna celda sin <code>data-label</code> .
El buscador	«tiza» deja 2 filas, «consumible» deja 5, «zzzqqq» deja 0 y aparece el aviso. Al borrar vuelve a 16.
La tecla Enter	Abre y cierra el menú en las tres páginas, y el botón recibe el foco. Con Enter y con clic.
Los botones de la tabla	Editar y Eliminar con clic real sobre filas distintas. Editar carga los cinco campos; Eliminar quita la fila.
Los cuatro mensajes	Los cuatro aparecen a la vez, pegados a su campo, con <code>aria-invalid</code> y <code>aria-describedby</code> . Cambian de texto según el valor y desaparecen al corregir.
Las reglas del dato	<code>-3</code> y <code>2.5</code> se rechazan en existencias; <code>0</code> se acepta. El precio <code>0</code> se rechaza y <code>15000</code> se acepta.
Validador W3C	0 errores y 0 avisos en las cuatro páinas.
Desbordamiento horizontal	Ninguno en 17 anchos entre 320 y 1920 px, en las tres páginas.
Auditoría del CSS	0 hexadecimales sueltos, 0 <code>rgb()</code> , 0 <code>!important</code> , 0 <code>@media (max-width)</code> , 0 <code>outline: none</code> , 0 <code>float</code> . Las 35 variables que se usan están definidas.

Evidencia

El enunciado pide un video de 60 segundos. **No puedo grabar video**, y en el día 6 quedamos en que no hacia falta ningún guion. Así que en lugar del video van estas cuatro capturas, tomadas del navegador con el ancho real de cada caso.

Socio: Snieder Alvarez						
Implementos y consumibles disponibles para alquiler						
TAC-001	Taco de pool 1,45 m	Implemento de juego	Nuevo	8	\$ 12.000	Editar Eliminar
BOL-004	Juego de bolas de billar	Implemento de juego	Nuevo	3	\$ 45.000	Editar Eliminar
TRI-002	Triángulo para bolas	Implemento de juego	Nuevo	5	\$ 8.000	Editar Eliminar
EXT-005	Extensión de taco	Implemento de juego	En reparación	1	\$ 10.000	Editar Eliminar
PUN-006	Puntería profesional	Implemento de juego	Nuevo	4	\$ 18.000	Editar Eliminar
SOP-016	Soporete para taco	Implemento de juego	Nuevo	15	\$ 5.000	Editar Eliminar
TIZ-010	Tiza de billar	Consumible	Nuevo	24	\$ 2.000	Editar Eliminar
GUA-003	Guantes de fibra	Consumible	Semi-nuevo	6	\$ 15.000	Editar Eliminar
GUA-007	Guantes de cuero sintético	Consumible	Semi-nuevo	7	\$ 22.000	Editar Eliminar
AGT-014	Agente de limpieza para mesas	Consumible	Nuevo	12	\$ 7.500	Editar Eliminar
TIZ-015	Tiza de Tournament	Consumible	Nuevo	20	\$ 4.500	Editar Eliminar
REP-011	Repuesto de punta de taco	Repuesto	Nuevo	18	\$ 3.500	Editar Eliminar
REP-012	Banda de goma para pocket	Repuesto	Nuevo	9	\$ 6.000	Editar Eliminar
CON-013	Conector de extensión	Repuesto	En reparación	2	\$ 9.000	Editar Eliminar
MES-008	Mesa de billar 8 pies	Mesa de billar	Nuevo	2	\$ 60.000	Editar Eliminar
MES-009	Mesa de billar 9 pies	Mesa de billar	Nuevo	1	\$ 85.000	Editar Eliminar

Punto 1. Las dieciséis filas pintadas desde `js/datos-prueba.js`, con sus botones de Editar y Eliminar. En el HTML no queda ninguna fila escrita a mano.

ario • Sneider Alvarez

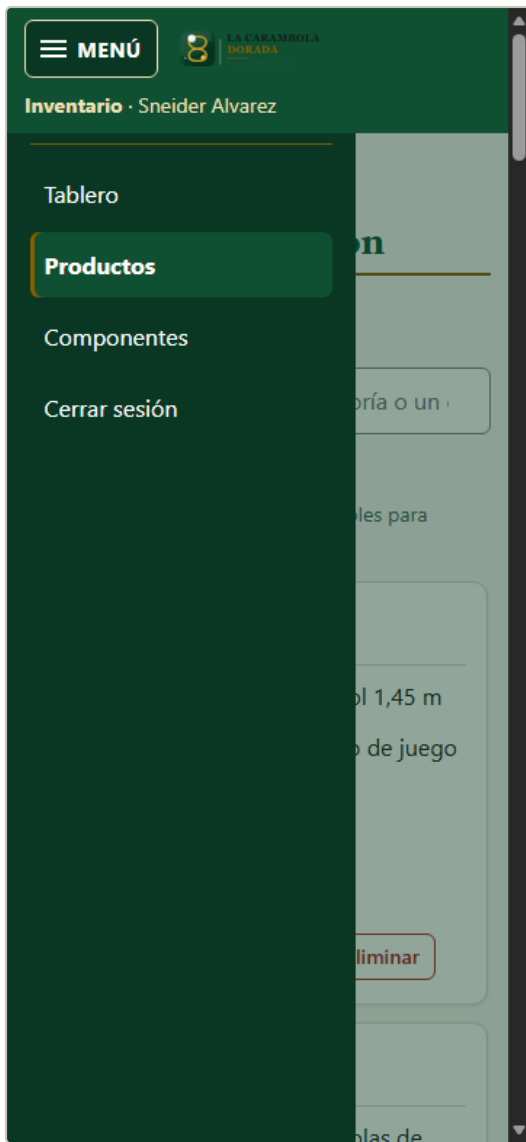
Buscar en el inventario

consumible

5 de 16 productos coinciden con la búsqueda.

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	VALOR DE ALQUILER	ACCIONES
Implementos y consumibles disponibles para alquiler						
TIZ-010	Tiza de billar	Consumible	Nuevo	24	\$ 2.000	Editar Eliminar
GUA-003	Guantes de fibra	Consumible	Semi-nuevo	6	\$ 15.000	Editar Eliminar
GUA-007	Guantes de cuero sintético	Consumible	Semi-nuevo	7	\$ 22.000	Editar Eliminar
AGT-014	Agente de limpieza para mesas	Consumible	Nuevo	12	\$ 7.500	Editar Eliminar
TIZ-015	Tiza de Tournament	Consumible	Nuevo	20	\$ 4.500	Editar Eliminar

Punto 2. El buscador filtrando en vivo por «consumible»: de dieciséis productos quedan cinco, y el contador de arriba lo anuncia.



Punto 3. El menú abierto a 360 px, desde el `<button>` que ahora es un control de verdad y responde al teclado. Esta imagen es de la versión con el menú desplegable, que solo existe por debajo de 1024 px.

Punto 4. Los cuatro mensajes de validación juntos, cada uno debajo de su campo: nombre, categoría, existencias y valor de alquiler. Ningún `alert`.

Cómo se tomaron estas capturas, para que quede claro: se abrió Edge con las medidas reales de cada caso, se mandó la tecla Enter y los clics por el protocolo del navegador, y se comprobó el estado de la página en el instante de la captura: las 16 filas, las 5 filtradas, el menú con `aria-expanded="true"` y visible, y los cuatro textos de error. Las capturas son del navegador; no las dibujé yo.

Lo que no hice, y por qué

- **El video de 60 segundos.** No puedo grabar. Si hace falta, se graba a mano con la grabadora del celular siguiendo los cuatro puntos: filtrar en vivo, abrir el menú con Enter y disparar los cuatro mensajes.
- **Botón de cancelar la edición.** No lo pide el enunciado. Mientras se edita un producto y se quiere registrar otro, hay que recargar la página. Es una carencia real del prototipo, no un olvido.
- **La tecla Escape para cerrar el menú.** Se puede cerrar con Enter en el botón Cerrar y con clic en el velo, que es lo que pedía el punto 3. Escape sería un extra.
- **Guardar en la base de datos.** El formulario sigue apuntando a `productos.php`, que todavía no existe. Los cambios se guardan en memoria y se pierden al recargar, que es lo propio de un prototipo.

- **Un aviso para quien no tiene JavaScript.** Sin JavaScript la tabla queda vacía, porque el punto 1 pide expresamente que no haya filas escritas en el HTML. Un `<noscript>` avisando de eso quedaría bien, pero es algo que el enunciado no pide.

Sneider Alvarez Santos

Aprendiz — Ficha 3428392

Recepción / Instructor

La Carambola Dorada — Actividad de recuperación día 8

Sneider Alvarez Santos · 1016717046 · ficha 3428392